

EEE3096S Practical 2

Phila Haanyama
HNYPHI001

Lulama Lingela
LNGLUL002

I. INTRODUCTION

The aim of this practical was to use Assembly code for the STM32 microcontroller to gain experience with low-level embedded programming. Assembly requires explicit manipulation of registers and memory addresses, which led to a deeper understanding of the microcontroller's architecture that was not needed when previously using the high-level language C. The primary objectives were to design control logic for LED sequencing in response to pushbutton inputs, and implement time delays.

Specifically, the task involved meeting five functional requirements:

- 1) Incrementing the LED display by one every 0.7 seconds.
- 2) Incrementing by two when SW0 is pressed.
- 3) Reducing the delay to 0.3 seconds when SW1 is pressed.
- 4) Displaying a fixed pattern (0xAA) while SW2 is pressed and then restoring the previous count upon release.
- 5) Freezing the current pattern displayed LED while SW3 is pressed.

II. METHODOLOGY

To complete the practical, the following steps were undertaken in a structured and iterative manner:

1) Project Setup

The provided course repository was cloned from GitHub and imported into STM32CubeIDE. This ensured that all base configuration files and peripheral setups were in place for assembly-only development.

2) Algorithm Design

Before coding, algorithms and pseudocode were written for each task. The tasks were divided into smaller independent modules, which made it easier to split work between team members, implement incrementally, and debug effectively.

3) Delay Function Implementation

A software delay loop was designed to control timing. The number of iterations required for the delays was calculated using the STM32F0 system clock (8MHz). Each loop iteration was estimated to take four cycles (1 for SUBS, 3 for branching [1]), giving approximately:

$$\text{loops} = \frac{\text{delay} \times f_{\text{clk}}}{\text{cycles per iteration}}$$

For the required delays:

$$\text{loops}_{0.7s} = \frac{0.7 \times 8 \times 10^6}{4} \approx 1.4 \times 10^6,$$

$$\text{loops}_{0.3s} = \frac{0.3 \times 8 \times 10^6}{4} \approx 6.0 \times 10^5$$

These values were defined as constants and used in the delay loop. Task 1 was first implemented using the long delay, and Task 3 extended this by switching to the short delay whenever SW1 was pressed. Each stage was built and tested on the hardware for verification.

4) LED Increment Adjustment (Task 2)

Logic was added to detect SW0 and adjust the LED increment from +1 to +2 while pressed. This was implemented and debugged separately before integration.

5) Forced Preset LED Pattern (Task 4)

The system was extended to detect SW2. When pressed, the LED output was forced to display 0xAA, and upon release, the normal counting sequence resumed. The function was tested independently to ensure correct override behaviour.

6) LED Freeze Function (Task 5)

For SW3, a freeze function was implemented. When pressed, the LED output remained fixed, resuming normal operation only once the button was released. This was tested incrementally before integration.

7) Integration and Refinement

All tasks were integrated into a single logical flow, with priority ordering for button actions. The combined system was then thoroughly tested on the board. Code comments were added to improve readability and explain the purpose of each section.

III. RESULTS & DISCUSSION

The implementation successfully achieved all the specified requirements by combining GPIO configuration, polling-based input detection, and conditional branching which can be seen in Figure 1.

The GPIOA inputs were configured with pull-up resistors meaning a logic level 0 indicated a pressed button, while GPIOB pins were set as outputs to drive the LEDs. A variable stored in register R2 maintained the current LED pattern, which was updated or frozen depending on the button states. The behaviour of each functional requirement can be described as follows:

• Increment logic (default SW0)

Conditional branching arithmetic was used so that under normal operation, the LEDs incremented by one each

cycle. However when SW0 was held down, the increment was then doubled to two each cycle.

- **Timing control (SW1)**

A delay loop based on subtraction and branching determined the cycle duration. The loop count was dynamically adjusted between a “long” delay (0.7 s) and a “short” delay (0.3 s) depending on the state of SW1, demonstrating a flexible use of loop structures for time control.

- **Preset pattern control (SW2)**

Edge detection logic was used to differentiate between press and release events of SW2. On press, the LED pattern switched to 0xAA, then while on release, the system restored to the previously saved pattern and resumed counting. This required two registers (R5 for the saved pattern and R6 for the LED’s previous state pattern). This highlighted the importance of state preservation in sequential logic systems.

- **Freeze control (SW3)**

When SW3 was pressed, the LED pattern was held constant by bypassing update logic until the button was released, demonstrating how conditional branching can override system changes, effectively creating a temporary “hold” state until the input was released.

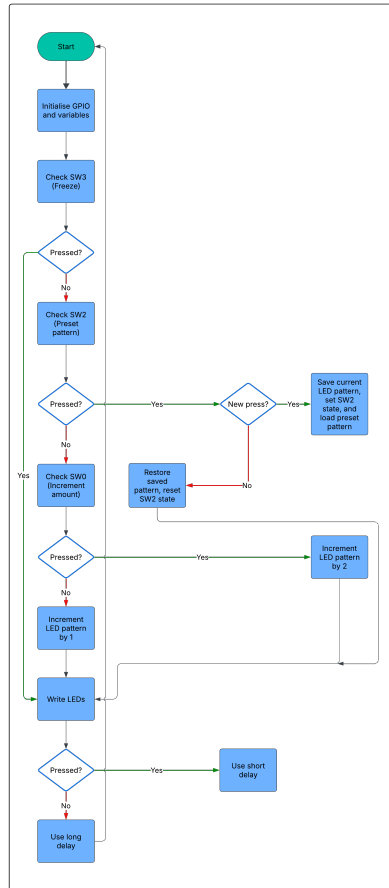


Fig. 1. Main control flow of the program

The program structure used a polling loop with dedicated sub-sections for each switch. While this method is effective for simple embedded systems, it is CPU-intensive since the processor spends most of its time going through loops.

Despite this limitation, the practical effectively demonstrated direct register manipulation, branching logic, and state tracking at the Assembly level. The successful implementation of all functional requirements confirmed the reliability of the designed control structure. The exercise also emphasised how multiple registers can be coordinated to manage states, preserve system history, and achieve predictable output behaviour.

IV. CONCLUSION

This practical provided insight into programming a micro-controller at the Assembly level. All required functionalities were implemented successfully using structured branching logic and register operations.

Future implementations could involve optimising the delay loop for efficiency, implementing debouncing for greater robustness, or integrating the use of interrupts instead of polling, which would reduce CPU load and make the system more responsive. In real-world applications, interrupts would prove to be more efficient as it would allow the processor to react only when an event occurs.

Overall, the practical reinforced a clear understanding of working with registers and low-level logic, forming a strong foundation for understanding how higher-level embedded code is executed on hardware.

V. AI CLAUSE

During this practical, large language models (LLMs) were used as a supportive tool to improve efficiency and understanding. They provided high-level explanations of ARM assembly concepts and GPIO operation, which helped clarify the requirements of each task. When assembler errors occurred, the LLM offered detailed interpretations of the messages and suggested possible fixes, which we then tested and validated on the STM32 hardware. Additionally, AI was used to resolve LaTeX formatting issues while compiling the report. Overall, LLMs served as an effective supplement for debugging, learning, and presentation, but the actual implementation, testing, and verification of the assembly code was carried out independently.

REFERENCES

- [1] Arm Developer. (2025) Instruction set summary, cortex-m0 technical reference manual r0p0. Accessed: Aug. 19, 2025. [Online]. Available: <https://developer.arm.com/documentation/ddi0432/c/programmers-model/instruction-set-summary>

Appendix

EEE3096S 2025 Practical 2 Code

Phila Haanyama (HNYPHI001) & Lulama Lingela (LNGLUL002)

Github Repo Link: <https://github.com/LlamaLuLu/EEE3096S-Practicals-Group-34.git>

```
/*
 * assembly.s
 *
 */

@ DO NOT EDIT
.syntax unified
.text
.global ASM_Main
.thumb_func

@ DO NOT EDIT
vectors:
    .word 0x20002000
    .word ASM_Main + 1

@ DO NOT EDIT label ASM_Main
ASM_Main:

    @ Some code is given below for you to start with
    LDR R0, RCC_BASE          @ Enable clock for GPIOA and B by setting bit 17 and 18 in
RCC_AHBENR
    LDR R1, [R0, #0x14]
    LDR R2, AHBENR_GPIOAB    @ AHBENR_GPIOAB is defined under LITERALS at the end of the
code
    ORRS R1, R1, R2
    STR R1, [R0, #0x14]

    LDR R0, GPIOA_BASE        @ Enable pull-up resistors for pushbuttons
    MOVS R1, #0b01010101
    STR R1, [R0, #0x0C]
    LDR R1, GPIOB_BASE @ Set pins connected to LEDs to outputs
    LDR R2, MODER_OUTPUT
    STR R2, [R1, #0]
    MOVS R2, #0               @ NOTE: R2 will be dedicated to holding the value on the LEDs

@ TODO: Add code, labels and logic for button checks and LED patterns
MOVS R5, #0                  @ initialize saved pattern
MOVS R6, #0                  @ initialize SW2 previous state (0 = not pressed)

main_loop:
    @ ----- check inputs -----
    LDR R3, [R0, #0x10]      @ load IDR into R3

    @ ----- freeze control (SW3) -----
check_SW3:
    MOVS R4, #8              @ load bit 3 into R4
    TST R3, R4               @ bitwise AND
    BEQ SW3_pressed
    @ SW3 not pressed:
```

B check_SW2 @ continue

SW3_pressed:

@ just freeze (don't change R2)

B write_LEDs @ skip all logic

@ ----- preset LED pattern control (SW2) -----

check_SW2:

MOVS R4, #4 @ load bit 2 into R4

TST R3, R4 @ bitwise AND

BEQ SW2_currently_pressed

@ SW2 not currently pressed (check if just released)

CMP R6, #1 @ SW2 previously pressed?

BNE skip_restore @ if already not pressed, don't restore

@ SW2 just released (restore saved value)

MOVS R2, R5 @ restore saved pattern

MOVS R6, #0 @ set to "not pressed"

B write_LEDs @ skip increment

skip_restore:

@ SW2 now not pressed (normal increment)

MOVS R6, #0 @ set to "not pressed"

B check_SW0 @ increment logic

SW2_currently_pressed:

@ check for new press (edge detection)

CMP R6, #0 @ SW2 previously not pressed?

BNE skip_save @ if already pressed, don't save again

@ 1st press (save current incrementing pattern)

MOVS R5, R2 @ save current pattern

MOVS R6, #1 @ set to "pressed"

skip_save:

@ while SW2 pressed (load preset pattern)

MOVS R2, #0xAA @ load preset LED pattern

B write_LEDs @ skip increments

@ ----- LED increment logic (SW0: while SW2 not pressed) -----

check_SW0:

MOVS R4, #1 @ load bit 0 into R4

TST R3, R4 @ bitwise AND

BEQ SW0_pressed

@ if SW0 not pressed:

ADDS R2, R2, #1 @ increment LEDs by 1

B write_LEDs

@ if SW0 pressed:

SW0_pressed:

ADDS R2, R2, #2 @ increment LEDs by 2

@ ----- output LEDs -----

write_LEDs:

STR R2, [R1, #0x14] @ write to LEDs

@ ----- delay logic (SW1) -----

check_SW1:

MOVS R4, #2 @ load bit 1 into R4

```

TST R3, R4                                @ bitwise AND
BEQ SW1_pressed
@ if SW1 not pressed:
LDR R4, LONG_DELAY_CNT                    @ load long delay
B delay_loop

```

```

@ if SW1 pressed:
SW1_pressed:
    LDR R4, SHORT_DELAY_CNT                @ load short delay

```

```

delay_loop:
    SUBS R4, R4, #1                        @ subtract 1 from R4, update flags (1 cycle)
    BNE delay_loop                        @ loops until count = 0 (3 cycles)

```

```

B main_loop

```

@ LITERALS; DO NOT EDIT

```

    .align
RCC_BASE:                .word 0x40021000
AHBENR_GPIOAB:           .word 0b11000000000000000000
GPIOA_BASE:               .word 0x48000000
GPIOB_BASE:               .word 0x48000400
MODER_OUTPUT:             .word 0x5555

```

@ TODO: Add your own values for these delays

```

LONG_DELAY_CNT:          .word 1400000 @ loops = 0.7 s * 8 MHz / 4 cycles
SHORT_DELAY_CNT:         .word 600000  @ loops = 0.3 s * 8 MHz / 4 cycles

```